

RESEARCH ARTICLE

Feature-Based Fault Localization in Evolving Software: Leveraging Regression Testing Insights

FAEZE AGHAZADE-PAR^{ID} AND MOJTABA VAHIDI-ASL^{ID}

Faculty of Computer Science and Engineering, Shahid Beheshti University, Tehran 1983969411, Iran

Corresponding author: Mojtaba Vahidi-Asl (mo_vahidi@sbu.ac.ir)

ABSTRACT Fault localization remains a vital yet resource-intensive task, particularly within software evolution, where swift and accurate fault localization is crucial. Whereas substantial research has improved fault localization techniques, challenges persist specifically in evolving software systems. Simultaneously, regression testing has been investigated to enhance testing efficiency while maintaining fault localization accuracy. Despite these advancements, further methodological improvements are necessary to refine fault localization accuracy in evolving software environments. This study proposes a new combination of features extracted from the source code and the associated test suite. The features include coverage, data and control dependencies, and code changes, with a new feature, “Test Case Weight,” derived from the test suite. “Test Case Weight” incorporates the effects of regression testing, which is influenced by two regression testing techniques: test case reduction and selection. Consequently, this feature is entirely influenced by regression testing. The results show that the proposed method achieves over 81% fault localization in the *Top-3* rankings. It also improves *EXAM* score, with over 68% of programs scoring 10 or less via test case selection and 63% via test case reduction. Compared to spectrum-based techniques, the proposed method located 54% more faults than *Tarantula* and *Ochiai*, and 45% more than *Jaccard* in the *Top-3* rankings. Additionally, it outperforms existing methods in *EXAM* score while using only 40% to 80% of the test suite. These findings highlight the importance of incorporating historical data, code structure, runtime behavior, and test case to improve fault localization efficiency in evolving software.

INDEX TERMS Fault localization, regression test, test case selection, test case reduction.

I. INTRODUCTION

Software is constantly evolving and receiving updates, which increases its complexity and makes testing more challenging due to the limited time and resources during the software development lifecycle. Various methods for fault localization have been proposed to identify faults quickly and accurately. One popular technique is spectrum-based fault localization (SBFL), known for its simplicity, cost-effectiveness, and speed. As a result, numerous formulae and approaches inspired by SBFL have been developed, with *Tarantula* [1], *Jaccard* [2], *Ochiai* [2], and *D-Star* [3] being among the most widely utilized. A substantial body of research focused on SBFL aims to enhance these techniques by either modifying

the formula used to calculate the suspiciousness score or adjusting the granularity level. However, these approaches have drawbacks, including the “tie” problem, where multiple statements may receive the same score, making it challenging to determine the faulty ones [4], [5], [6], [7].

Machine learning and deep learning techniques have also emerged as effective fault localization methods by extracting features from code. Nevertheless, these techniques can be less efficient when applied to larger programs and require extensive datasets for training [8], [9]. Studies indicate that, in addition to the limited accuracy of existing fault localization methods, there is a lack of research in fault localization for evolutionary programs. This type of fault localization aims to identify the location of faults introduced by changes that regression tests have uncovered. Besides, as test cases are crucial in software testing, they play a similar role in fault

The associate editor coordinating the review of this manuscript and approving it for publication was Yang Liu^{ID}.

localization. However, a significant issue with existing fault localization techniques is their neglect of the importance and variations of test cases [10].

Moreover, despite extensive research on regression testing, many of which emphasize fault localization accuracy, studies in fault localization rarely integrate regression testing methodologies alongside fault localization principles. There is a notable gap in approaches that concurrently consider both domains, leveraging the insights and artifacts of regression testing to enhance the fault localization effectiveness. Consequently, this research aims to develop a fault localization method that can accurately locate faults by leveraging more source code and version history information while directly incorporating regression testing as both the technique and the feature.

The proposed method utilizes 1) Coverage as a straightforward and effective feature for both fault localization and regression testing; 2) Dynamic Data and Control dependencies as features that represent runtime behavior of the code and complement coverage to address the “tie” problem; 3) Change as a feature that accounts for code modifications, impacting both fault localization and regression testing processes; and 4) Test Case Weight, which integrates regression testing directly into fault localization by evaluating test cases based on their effectiveness in locating faults and differentiates program statements based on the role of the test cases that covered the statements in fault localization. Additionally, it employs regression testing techniques, namely Test Case Reduction (TCR) and Test Case Selection (TCS), to both prune test cases and minimize testing time while enhancing the accuracy of fault localization. Test Case reduction minimizes the test suite by identifying redundant or irrelevant tests within it [11], and Test Case Selection process focuses on identifying and selecting test cases relevant to software changes through impact analysis [12].

Accordingly, this study demonstrates that fault localization and regression tests are more interrelated than commonly perceived, highlighting features that can benefit two distinct but related processes. Consequently, it introduces “Test Case Weight” as a new concept and feature. Furthermore, the study demonstrates the limitations of relying solely on spectrum information for fault localization and emphasizes the necessity for more informative data derived from the code. In contrast to other existing research, this study explicitly addresses faults arising while implementing new software versions. In summary, this study contributes the following:

- It emphasizes software evolution by incorporating “Change” feature values as a critical element in fault localization.
- It introduces “Test Case Weight” as a new feature for calculating suspiciousness scores, demonstrating its effectiveness in fault localization.
- It integrates regression testing with fault localization through the “Test Case Weight” feature.

- The formula is evaluated on real-world programs of varying sizes and complexities, each containing real fault(s).

The structure of this paper is outlined as follows: Section II presents an illustrative example to elucidate the core concept of the proposed methodology. Section III delves into the review of pertinent literature. Section IV provides a detailed exposition of the proposed approach. Section V describes the evaluation framework and experimental design. Subsequently, section VI presents the results of the experiments. Finally, the last two sections are reserved for discussion and conclusions.

II. MOTIVATIONAL EXAMPLE

In this section, a simple example illustrates the proposed method’s functionality. Subsequently, we repeated the same example using SBFL techniques to highlight their limitations and underscore the proposed method’s advantages.

SBFL is a lightweight method that summarizes coverage information for each program element across a test suite into a spectrum and computes a suspiciousness score from it, indicating the likelihood of statements having a fault [13]. Specifically, a faulty program is instrumented and executed with a test suite in spectrum-based techniques. For each execution, a spectrum is generated that captures the program’s dynamic behavior during that execution, encompassing coverage data and test execution results. This spectrum facilitates the assessment of code sections (such as statements, branches, or functions) in both failed and successful executions. A ranking metric is then applied to evaluate the suspiciousness level and rank the program elements. In particular, the probability of a code element being faulty increases if it is frequently encountered in failed executions, whereas appearing less often in passed ones [5]. Given the popularity of these techniques, extensive research has been conducted to modify formulae or adjust granularity. Here, we also mention three widely used formulae used in this research. These formulae utilize the following notation:

N_{cf} : Number of failed runs in which the statement is covered

N_{cf} : Number of failed runs in which the statement is not covered

N_{cp} : Number of passed runs in which the statement is covered

N_{cp} : Number of passed runs in which the statement is not covered

Suspiciousness score (T)

$$= \frac{N_{cf} / (N_{cf} + N_{uf})}{(N_{cf} / (N_{cf} + N_{uf})) + (N_{cp} / (N_{cp} + N_{up}))} \quad [1] \quad (1)$$

Suspiciousness score (O)

$$= N_{cf} / \sqrt{(N_{cf} + N_{uf}) \cdot (N_{cf} + N_{cp})} \quad [2] \quad (2)$$

Suspiciousness score (J)

$$= N_{cf} / (N_{cf} + N_{uf} + N_{cp}) \quad [2] \quad (3)$$

TABLE 1. Example of SBFL formulae.

Statements of Code	N_{cf}	N_{uf}	N_{cp}	N_{up}	Tarantula	Ochiai	Jaccard
num = int(input())	6	0	4	0	0.5	0.301511	0.6
factorial = 0 //fault	6	0	4	0	0.5	0.301511	0.6
if num < 0:	6	0	4	0	0.5	0.301511	0.6
print("Sorry, factorial does not exist for negative numbers")	0	6	3	1	0	0	0
else if num == 0:	6	0	1	3	0.8	0.353553	0.857143
print("The factorial of 0 is 1")	0	6	1	3	0	0	0
else:	6	0	0	4	1	0.377964	1
for i in range(1,num + 1):	6	0	0	4	1	0.377964	1
factorial = factorial*i	6	0	0	4	1	0.377964	1
print("The factorial of", num, "is", factorial)	6	0	0	4	1	0.377964	1

TABLE 2. Example of the proposed method.

Statements of Code	C	D1	D2	C1	C2	CH	TCW	TCS Score	TCR Score
num = int(input())	5	20	0	0	0	0	5	-2.596	-0.207
factorial = 0 //fault	5	5	0	0	0	1	5	-0.947	1.153
if num < 0:	5	0	5	5	0	0	5	-2.671	-0.207
print("Sorry, factorial does not exist for negative numbers")	0	0	0	0	0	0	0	-3.111	-0.707
else if num == 0:	5	0	5	5	5	0	5	-2.626	-0.257
print("The factorial of 0 is 1")	0	0	0	0	0	0	0	-3.111	-0.707
else:	5	0	0	10	5	0	5	-2.701	-0.467
for i in range(1,num+1):	5	10	10	5	5	1	5	-0.767	1.383
factorial = factorial*i	5	9	14	0	5	0	5	-2.449	0.122
print("The factorial of", num, "is", factorial)	5	0	10	0	5	0	5	-2.551	-0.047

Table 1 presents an example of 10 test cases analyzed using these formulae. The table shows that the SBFL formulae fail to differentiate between various statements, resulting in multiple statements receiving identical suspiciousness scores. Consequently, the faulty statement shares the same score as the two other statements. Notably, all three formulae rank the faulty statement as the 7th most suspicious statement to have the fault.

Table 2 presents an example of the proposed method involving test case selection and reduction, utilizing only five test cases. The detailed calculations for this example are outlined step by step in Section IV. For clarity, we will employ specific notations in all tables, which will also be referenced in the subsequent section on the proposed method.

C: Coverage

D1: Data dependency (*def*)

D2: Data dependency (*use*)

C1: Control dependency (*dom*)

C2: Control dependency (*domed*)

CH: Change

TCW: Test Case Weight

As illustrated in Table 2, the suspiciousness scores for each distinct statement vary due to the proposed combination of selected features employed in the calculation, that result in unique statements from a fault localization perspective. This variability highlights the proposed method's effectiveness in addressing the "tie" problem commonly associated with SBFL. Furthermore, the faulty statement is ranked in the top two for test case reduction and selection, indicating that the proposed method not only achieves higher accuracy compared to SBFL but also results in a reduced test suite size due to the use of TCR/TCS.

III. RECENT WORKS

Our proposed method encompasses fault localization and regression testing; therefore, we surveyed recent works in these two fields. A research introduced a notable method, *FLITSR*, to enhance the spectrum-based approach for improved performance in multi-fault environments. This method employs iterative test case reduction to achieve more accurate fault localization. Specifically, it tests once to identify a single fault. Subsequently, it utilizes test case

reduction to eliminate test cases biased toward the coverage of the identified fault, thereby shifting the focus of testing toward discovering new faults [13]. Similarly, another study proposed a solution to enhance spectrum-based techniques in multi-fault environments. This approach introduces an algorithm inspired by coverage-based test case reduction. This method re-rank segments of code deemed non-faulty (i.e., those downgraded by spectrum-based analysis), reducing their likelihood of being overlooked. The authors employed Multiverse Analysis to simulate potential repairs instead of re-running all test cases to identify all faults. This technique allows for an estimation of fault localizations and enables adjustments to the fault localization strategy accordingly. The core idea is to simulate various scenarios to uncover all faults within the program without extensive re-testing.

Another study [14] also presented a deep learning approach called *FixLocator*, which identifies faulty statements across multiple methods that may require simultaneous fixing. This method utilizes two models: one operating at the method level to learn which methods are interconnected and need concurrent repairs and another at the statement level to pinpoint specific statements that should be addressed together. A novel feature termed “co-changed statements” is introduced, suggesting that statements with common modifications may necessitate future fixes. Furthermore, the authors employ a graph convolutional network to integrate various dependencies, including data and control dependencies, execution traces, stack traces, and test case coverage. Research [15] proposed a framework for fault localization and general coverage calculation for Domain-Specific Languages (DSL). In this method, the coverage matrix is computed based on three inputs: 1) DSL specification, 2) coverage rules, and 3) model execution trace. Utilizing the results from test case execution and the calculated coverage matrix, the framework performs a ranking based on a suspiciousness score.

Moreover, a study examined several families of large-scale fault localization methods applied to real Python programs and faults [16]. This study investigated the effectiveness and efficiency of seven popular fault localization techniques from four families: 1) spectrum-based, 2) mutation-based, 3) predicate switching, and 4) stack trace analysis. The methods *Tarantula*, *D-Star*, and *Ochiai* were employed within the spectrum-based category, whereas *Metallaxis* and *Muse* were utilized in the mutation-based category. The researchers also explored combinations of all seven methods in two different configurations to assess the effectiveness of these combinations. The results indicate that spectrum-based techniques are the most effective, followed by mutation-based techniques. Furthermore, whereas SBFL and Mutation-Based Fault Localization (MBFL) often demonstrate similar effectiveness, SBFL performs better when fewer faults are present. The study also discovered that combining techniques enhances effectiveness but does not improve efficiency.

Similarly, another study [17] aimed to investigate the effectiveness of spectrum-based formulae on real Python projects. The spectrum-based formulae examined included *Tarantula*, *Barinel*, *Ochiai*, *Op*, and *D-Star*. The results of this study indicate that older methods (*Tarantula*, *Ochiai*, and *Barinel*) outperform newer approaches. Additionally, the performance of spectrum-based formulae does not vary based on the type of fault; instead, results show that performance on multi-line faults is superior to that on single-line faults. Moreover, [18] proposed an Abstract Syntax Tree (AST-based) fault localization method that employs a graph neural network called *GraphSAGE*. This network first learns the features of each node based on the AST structure and coverage information. Then, it uses a multilayer perceptron to calculate the suspiciousness of each statement. Similarly, [19] presented a two-level approach for fault localization based on a graph neural network (GNN). This GNN extracts data and control dependency information as well as data and control flow. In the first phase, it calculates the suspiciousness of each method and determines its rankings. In the second phase, it identifies potential fault locations within a method using a fine-grained graph neural network.

Additionally, faults may occur when two separate pieces of code are executed simultaneously, and spectrum-based methods may not detect this connection. To address this limitation, [20] proposed a method for learning the local relationships between code segments and test case outputs. They employed a spectrum-based approach as a classification model to predict whether a test case will pass or fail based on the spectral information it provides. Furthermore, they utilized eXplainable Artificial Intelligence (XAI) techniques to interpret each code segment’s local importance in predicting each test case’s outcome. This design enables the automatic learning of how failed test cases contribute to the suspiciousness of various parts of code. In addition, [21] introduced a feature-based fault localization method known as Feature-FL. This method integrates feature diversity from the perspective of program features with suspiciousness assessment. It considers the “probability of branch execution” as a feature and employs feature selection to establish the relationship between each feature and program failures. Ultimately, it associates each feature with its corresponding statement and utilizes this relationship to determine suspiciousness.

Furthermore, Dutta proposed a technique that utilizes statement frequency information while considering the contribution of individual test cases to assign a suspiciousness score to each statement and to generate a ranked list of test cases [12]. This method also accounts for the variance among test cases. The study analyzed the internal relationships between program execution information and test case execution results using conditional probability models derived from quantitative statistics. The suspiciousness score is then calculated based on these probabilities. In this approach, the relationship between test case results and statement coverage is considered similarly to our “Test Case Weight” feature.

However, the utilization of this information and the corresponding calculations differ significantly. Furthermore, this method relies exclusively on this data without incorporating broader contextual factors, such as those utilized in SBFL.

In conclusion, despite the extensive research conducted on fault localization, there remains a significant challenge in developing methods that specifically address software versioning. Furthermore, much of the existing research overlooks the critical role of the testing process and test suites in effective fault localization, underscoring the relevance of the current study. In addition to fault localization, some recent research on regression testing techniques, including test case reduction and selection, are also presented.

Liu et al. conducted a research to refine Regression Testing Strategies (RTS) to expedite the regression testing process [11]. This method manually examines changes to identify those that do not necessitate re-testing and are solely affected by specific modifications. In other words, the premise is that a series of semantic code changes do not require re-executing all tests selected through analysis-based selection. Ghani et al. also proposed an improved and efficient test case selection algorithm for regression testing [22]. The proposed technique reduces execution time and eliminates redundant test cases by identifying only those changes relevant to the modifications made in the test cases. This technique incorporates both test case selection and change detection algorithms, utilizing the history of test case changes to calculate the number of modifications and the corresponding affected test cases. Furthermore, Ozkan et al. proposed a graph-based genetic algorithm to address the test selection problem in regression testing [23]. In this approach, each individual in the population is represented as a node within a predefined graph structure, and its crossover probability is constrained based on neighborhood relationships. This strategy aims to enhance population diversity, prevent premature convergence, and improve overall convergence performance. Practically, this method operates by identifying or selecting the minimum necessary test cases to optimize the performance of the genetic algorithm. By positioning the population on the graph and restricting crossover options to neighborhood relationships, this approach effectively increases diversity within the population.

In addition, [24] presented an approach that combines code changes and code smells to facilitate test case selection. They introduced two techniques: one based on changes and the other on code smells. This method effectively addresses faults caused by poor design and low software quality by incorporating code smells into the selection process. Moreover, Singhal et al. employed the African Buffalo Optimization Algorithm to prioritize and select test cases [25]. In their approach, each test case is represented by a “buffalo,” which moves randomly to create a path that it believes to be safe, meaning it covers all potential faults. The buffalo traverses the test cases, and if any of these test cases fail while covering faults, the buffalo “kills” them. The buffalo reaches a “safe place” after eliminating all identified faults. Once

all buffaloes have created their paths, they are compared based on execution time. Besides, Raju et al. proposed a method called *SCARF-RT*, which utilizes a similarity coefficient, abbreviation generation, regression testing, and fuzzy set theory to select test cases for a test suite [26]. This method employs ranking based on the similarity coefficient to cluster test cases. They implemented two techniques: 1) Each cluster contains a set of distinct features, and the selection of test case clusters is based on feature coverage, with a ranking used to form these groupings; and 2) Similarity between test cases is determined based on eleven features. They applied the max-min combination method to identify fuzzy equivalences, which inform the formation of clusters. They ultimately grouped the most similar test cases to construct the final test suite.

Additionally, Nagy et al. proposed a test suite reduction technique that employs clustering and a genetic algorithm [27]. They executed a reduced test suite in parallel to minimize execution time. This method first determines the optimal number of clusters using silhouette analysis. Second, K-medoid clustering is used to partition the test cases. Finally, the number of test cases within each cluster is reduced using the *NSGA-II* genetic algorithm. Besides, Singh et al. [28] proposed a test case reduction method for agile software development using support-based reduction and a whale optimization algorithm (WOA). Their research focused on identifying optimal test case clustering strategies for agile teams. The method comprises two stages: 1) Prioritization and Selection, a one-time execution-based selection of critical test cases, and 2) the WOA, which selects test cases based on high failure rate or coverage. Pan et al. [29] introduced a scalable closed box test suite reduction approach leveraging large language models (LLMs) to maintain high fault detection rates. Their method explores five pre-trained LLMs and employs cosine similarity and Euclidean distance to measure test case similarity. The goal is to identify computationally efficient similarity metrics that effectively guide a genetic algorithm for optimal test suite reduction.

Studies show that reduction and selection techniques for test suites commonly employ diverse coverage metrics as primary algorithmic criteria, alongside execution time and code changes. These studies also demonstrate the effectiveness of critical test case selection in improving fault localization. However, although all regression testing techniques aim to expose failures and assist in fault localization, they are seldom directly incorporated into the fault localization process.

IV. PROPOSED APPROACH

This section details the proposed fault localization method, explaining the extraction, preprocessing, and application of features, including coverage, dynamic data and control dependencies, code changes, and test case weights. The formula incorporating these features is also derived. Furthermore, the section discusses the integrated approach of test case selection and reduction within the fault localization

process, outlining the selection and reduction criteria. Fig. 1 illustrates a high-level architecture of the method.

This combination of features is suggested based on several distinct research studies. Coverage is a widely used feature in all SBFL techniques and most other fault localization techniques. It is also a popular and effective feature in

regression testing. Data and Control dependencies are additional features that can help address the tie problem encountered in SBFL techniques, as statements with the same coverage may have different dependencies that SBFL cannot detect. The importance of these features has been studied by [30], [31], and [32].

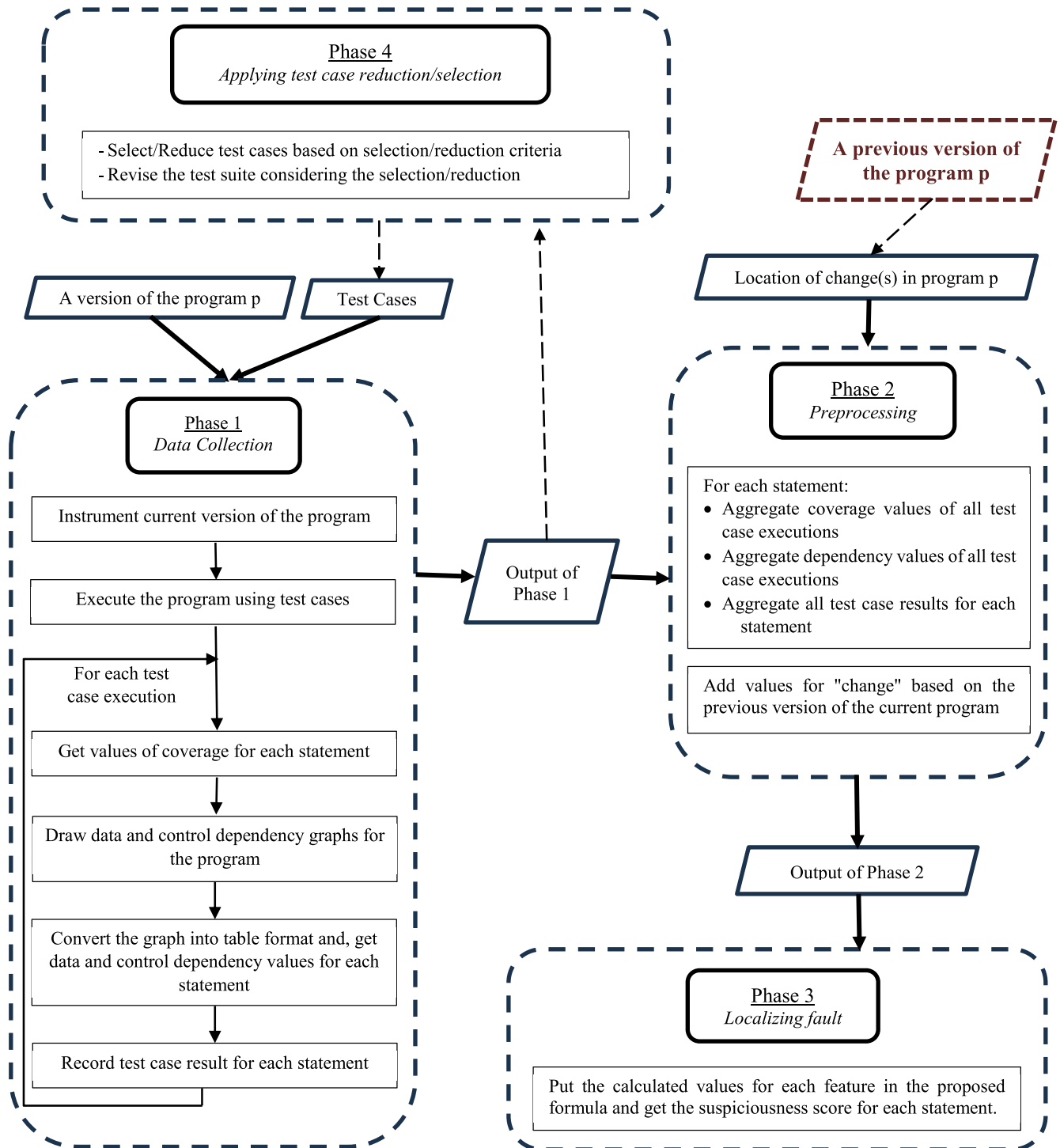


FIGURE 1. Architecture and methodology of the proposed method.

Furthermore, any change in code could result in a fault. In other words, the modified code sections are more likely to contain faults. As for “Test Case Weight”, this is a novel concept and draws inspiration from regression testing. In regression testing, certain test cases are preferred over others based on various valued attributes, such as test case coverage, their ability to reveal faults, execution time, etc. This implies that each test case impacts the testing process differently. Moreover, all testing techniques aim to reveal failures as early as possible. Therefore, test cases can also be a valuable feature in fault localization, making it sensible to evaluate statements based on the test cases that cover them. Thus, we introduced a novel technique that maintains the simplicity and clarity of SBFL methods while incorporating both program changes and regression testing techniques and results as key features for fault localization in evolving software systems.

Since “Test Case Weight” is a novel feature, it is important to define its concept. “Test Case Weight” is calculated as the cumulative results from test cases associated with a specific statement. When a program is executed with various test cases, some statements may not be executed at all, whereas others may be executed in passed and failed test cases. Each test case yields either a failure or a pass, with failed results assigned a value of 1 and passed results assigned a value of -1. This approach is based on the premise that failed results are more informative for fault detection than passed results; thus, statements covered by numerous passed cases are likely to be non-faulty. Consequently, the value of the corresponding “Test Case Weight” should decrease, which explains the assignment of -1 for passed test cases and 1 for failed test cases. To determine the “Test Case Weight” for each statement, we examine all executions in which the statement is covered, sum the results (1 for failures and -1 for passes), and derive the overall behavior of the statement throughout the testing process. In other words, this feature reflects the contribution of each statement. Unlike coverage metrics that assess a statement’s contribution based solely on execution, “Test Case Weight” values the statement based on both its contribution and the type of execution (failed or passed) in which it was contributed. Therefore, the composition of the test suite significantly influences “Test Case Weight values”, as the specific test cases determine the nature of the execution outcomes.

To provide a brief overview of the method, the inputs consist of 1) the current source code of the program, 2) the source code of the previous version of the program, and 3) the associated test suite. These inputs are also utilized for test case reduction and selection. Initially, the proposed method executes the current version of the program with the associated test suite. Next, data regarding coverage, dynamic data and control dependencies, and test case weights are gathered based on the execution of individual test cases. Notably, the test suite influences the test case weights directly and the other features indirectly. Subsequently, these feature values, alongside the modified sections of the code, are input into

a proposed formula to calculate a suspiciousness score for each statement. Finally, the proposed method rank statements according to their suspiciousness scores, with higher scores indicating a greater likelihood of faults. Subsequently, the proposed method carry out a test case selection or reduction process to prepare the test suite for the subsequent program version and its testing.

A. PHASE 1: DATA COLLECTION

This phase focuses on data collection. Initially, we instrument the given program to gather coverage and dependency data. Subsequently, we collect the features individually for each test case execution. As each test case runs, a code coverage matrix is generated, wherein covered statements get a value of 1, and uncovered statements are marked with 0. For simplicity, we incorporate “Data Dependency” into this matrix by converting dependency graphs into a tabular format. Specifically, we add two columns for “Data Dependency”, labeled “*def*” and “*use*,” as well as two columns for Control Dependency, labeled “*dom*” and “*domed*.” The Data Dependency “*def*” column signifies a variable that acquires a new value. More specifically, the corresponding cell in the matrix is incremented by one if a variable is assigned a new value in a particular statement, and this newly defined value is subsequently utilized by other statement(s). Conversely, if a statement does not assign a value, the corresponding cell remains 0. In contrast, the Data Dependency “*use*” column denotes a variable that references a predefined value. In this case, the relevant cell in the matrix is also incremented by 1 if a predefined value is utilized within the statement. Like Data Dependency “*def*,” if a statement does not use any data or values, the corresponding cell will get a value of 0.

Control Dependency “*dom*” addresses situations where one statement dominates others, meaning that the execution of certain statements is contingent upon the execution of a specific statement. This particular statement thus serves as a dominating element in the code, whereas the contingent statements are referred to as dominated statements. Consequently, the cell related to the dominant statement in the matrix under “Control Dependency (*dom*)” is incremented by 1 for each statement it dominates. Likewise, the cells in the “Control Dependency (*domed*)” column for dominated statements are also incremented by 1 for each dominating statement. Consistent with the data dependencies, if a statement does not dominate any other statements, the corresponding cell in the “Control Dependency (*dom*)” column will contain a 0; similarly, if a statement is not dominated by any other statements, the corresponding cell in the “Control Dependency (*domed*)” column will be 0.

A column for “Test Case Weight” and a column for “Change” have also been integrated into the matrix. The results of each test case execution are recorded in the “Test Case Weight” column. For simplicity, if a statement is covered in a test run and that run results in a pass, the corresponding cell in the “Test Case Weight” column is assigned a value of -1. Conversely, if a statement is covered in a test

run that fails, the cell is assigned a value of 1. If the test case does not cover the statement, the cell related to the statement receives a value of 0. The assignment of -1 to passed test cases is justified by the notion that these test cases failed to expose any faults in the program. Similarly, a value of 1 is assigned to failed test cases for the same rationale. By incorporating “Coverage” as a feature, we assess the frequency with which a statement is executed, whereas the predefined assignments in the test case weight feature allow us to evaluate the contribution of each statement during executions. For instance, if a statement receives a positive value for its test case weight, it indicates that it is covered in failed runs more frequently than in pass runs, regardless of the overall frequency of coverage. Consequently, this feature enables the identification of statements that contribute more significantly to failed runs. To the best of our knowledge, no prior research has utilized this feature. Furthermore, the “Change” column facilitates a comparison between the current version of the program and its previous version. We assign a value of 1 to the relevant cell if a statement has been modified; otherwise, it is assigned a value of 0.

In this phase, we collect data distinctly from each test case. For instance, consider a source code consisting of 10 lines of code and five associated test cases. In this scenario, the matrix will contain 50 rows of data, as each of the 10 lines is executed 5 times, resulting in the collection of feature values for all five runs for each statement. Consequently, each statement will have five distinct values corresponding to the features, reflecting the repeated executions across the test cases. The output of this phase is presented in Table 3.

TABLE 3. Matrix of extracted values of features in Phase 1.

Statements	C	D1	D2	C1	C2	CH	TCW
1	1	1	0	0	0	0	1
2	1	1	1	0	0	0	1
3	1	1	1	0	0	0	1
4	1	2	0	0	0	0	1
5	1	1	2	3	0	0	1
6	1	4	1	0	1	0	1
7	1	0	1	1	1	0	1
8	1	0	0	0	1	0	1
9	1	0	1	1	1	0	1
10	1	9	2	0	1	0	1

B. PHASE 2: PREPROCESSING

The input for this phase consists of the collected data formatted as a matrix from Phase 1, where the output of Phase 1 represents raw data. During Phase 2, we conduct a calculation on the data. Specifically, we sum the values in each column for every statement, and record the resulting value as the final value for that particular column and statement.

To illustrate this calculation, consider statement S1, which is covered by five test cases. In the matrix from Phase 1, S1 appears five times, with each corresponding coverage cell containing a value of 1. In Phase 2, these values are aggregated, resulting in a total of 5 for the coverage column of S1, representing its overall contribution. In contrast to Phase 1, each statement in this phase is represented by a single value for each feature, which is the aggregation of the values from all executions. This calculation applies to all features except the “Change” feature, as we determine only once if a statement has changed. The results of Phase 2 are presented in Table 4. The aggregation process can be understood by comparing Tables 3 and 4.

TABLE 4. Preprocessed values of features as the output of Phase 2.

Statements	C	D2	D1	C2	C1	CH	TCW
1	22	0	22	0	0	0	-8
2	22	22	22	0	0	0	-8
3	22	22	22	0	0	0	-8
4	22	0	32	0	0	0	-8
5	22	32	22	0	66	0	-8
6	22	31	85	22	0	0	-8
7	22	22	0	22	12	0	-8
8	12	0	0	12	0	0	0
9	22	22	0	22	22	0	-8
10	22	44	158	22	0	0	-8

C. PHASE 3: LOCALIZING FAULT

The output from Phase 2 includes all the necessary data for fault localization using the proposed method. By substituting these values into the following (4) and (5), a suspiciousness score is computed for each statement in the program. Subsequently, the statements are ranked based on their suspiciousness scores, such that the statement with the highest score is deemed the most likely to contain a fault. The proposed method is compatible with both test case reduction and selection; therefore, this study examines each approach separately. As a result, we conducted two formulae and two distinct experiments based on the same methodology previously described. Equation (4) corresponds to the proposed method of test case reduction, whereas (5) is specifically for the method of test case selection.

Suspiciousness score of Sn

$$= C \times 0.005 + D1 \times 0.005 + D2 \times 0.031 + C1 \times (-0.011) + C2 \times (-0.01) + CH \times 1.435 + TCW \times 0.075 - 0.707 \quad (4)$$

Suspiciousness score of Sn

$$= C \times (-0.011) + D1 \times 0.006 + D2 \times 0.012 + C1 \times (-0.003) + C2 \times 0.009 + CH \times 1.739 + TCW \times 0.09 - 3.111 \quad (5)$$

D. PHASE 4: APPLYING TEST CASE REDUCTION/SELECTION

For the test case reduction process, we systematically reduce 20% of the test suite for each version. The selection of test cases for reduction primarily hinges on their similarity. Initially, we identify test cases with comparable coverage and results. Then, we eliminate those exhibiting identical dependencies, test case weights, and change attributes among these similar test cases. Consequently, we remove redundant test cases, which are deemed identical. We assign preference to passed test cases within this set of redundant cases. Failed test cases are more informative for fault localization, as they are certain to cover the fault and often provide clearer indications of its location. In contrast, passed test cases may not even execute the faulty code and can remain entirely unaware of its presence. Consequently, they contribute less direct evidence toward identifying the faulty code. Therefore, prioritizing failed cases is expected to yield more useful information for pinpointing the fault's location.

Moreover, for the test case selection process, we collect a dataset comprising four distinct features for each test case: 1) test case result, 2) code coverage, 3) fault coverage, and 4) change coverage. Although the three coverage features may appear similar, they effectively differentiate between test cases that otherwise seem alike. Among the four features, we prioritize the result and fault coverage slightly more than the others. This prioritization is grounded in findings from prior studies in the Test Case Selection (TCS) domain, where similar criteria have been shown to improve fault detection effectiveness. We adopt the same rationale for prioritizing test case outcomes, as in TCR approaches, to maximize the likelihood of revealing faults. Fault coverage is also a critical factor, since feature values for faulty statements can only be obtained when those statements are executed. In addition, code coverage is a widely used measure; selecting test cases with higher coverage both increases the number of dependencies and relations available for analysis, and creates a more challenging evaluation scenario for the proposed method. Finally, because our dataset includes a "Change" feature, we specifically include test cases that execute changed statements in order to assess the contribution of this feature to fault localization accuracy. When test case selection is applied to version 3 or later of the program, it indicates that selection has been conducted at least once prior, allowing for accumulating historical data on the test cases. Thus, we aggregate the values of the relevant features from the initial selection to the most recent one, and perform the selection for the current version based on these cumulative values.

V. EXPERIMENTAL DESIGN

We dedicate this section to the research process, encompassing data collection, formula generation, and evaluation.

A. EXPERIMENTAL SETUP

To generate the proposed formulae (4) and (5), we employed a dataset comprising 14 distinct programs, each with a

minimum of three versions. This dataset contains 61 source codes and 116 identified faults. As detailed in the preceding section, the initial step involved collecting and computing pertinent data for the dataset. The outcomes of these phases were fundamental in the subsequent development of the formula.

In this study, we utilized a machine learning technique, logistic regression, as the primary objective of the formulae is to classify code statements into two categories: faulty and non-faulty. To facilitate the research, we added a column titled "result" to the output matrix from Phase 2, which indicates whether a particular statement is faulty. Specifically, a value of 1 in the "result" cell signifies that the statement is faulty, whereas a value of 0 denotes that it is non-faulty. We designated the "result" as the dependent variable for training the regression model. In contrast, all other features, including "Coverage", "Data Dependency (*def-use*)", "Control Dependency (*dom-dominated*)", "Change", and "Test Case Weight", were treated as independent variables. Given the nature of the problem, we observed that the frequency of zeros in the dependent variable substantially exceeds that of ones, resulting in an imbalanced dataset. We examined various sampling techniques, including oversampling and undersampling methods to address this problem. Within the realm of oversampling, we investigated the *Synthetic Minority Over-sampling Technique (SMOTE)* and *Random OverSampler*. In contrast, we implemented methods such as *EditedNearestNeighbors* and *RandomUnderSampler* for undersampling. The exploration of these sampling techniques aimed to identify a method that would yield optimal results for the specific dataset. Once we balanced the dataset, we employed logistic regression for analysis. The model generated correlation coefficients for each independent variable, which were subsequently utilized to construct the final equations. **Throughout this experiment, we posed the following research questions:**

- Q1: What is the effect of "Test Case Weight" on fault localization when employing test case reduction?
- Q2: How does "Test Case Weight" affect fault localization when utilizing test case selection?
- Q3: To what extent has the accuracy of fault localization improved by integrating this combination of features?
- Q4: How does employing test case reduction directly affect fault localization?
- Q5: How does utilizing test case selection directly affect fault localization?

In summary, these inquiries comprehensively investigate the key features of the proposed method, emphasizing the role of code modifications and the significance of test case weights, in conjunction with coverage and dependencies, in enhancing the accuracy of fault localization.

B. DATA COLLECTION

All source codes and their corresponding test suites were sourced from the *Code4Bench* benchmark. *Code4Bench* features real-world code samples containing actual faults derived

from *Codeforces*, with most of these samples being brief and simple. Additionally, in smaller programs, the challenge of “tie” problems becomes increasingly pronounced. Thus, these programs present an excellent opportunity to illustrate both the advantages of the proposed method and the shortcomings of traditional Spectra-Based Fault Localization techniques. This benchmark also encompasses code samples exhibiting various types of faults, including Compilation errors, Wrong answers on tests, Time limit exceeded on tests, and Memory limit exceeded on tests, among others. For this study, we restricted our analysis to those codes that specifically resulted in “Wrong answers on tests.” The selected programs were written in Java; however, the proposed method also applies to other programming languages. Moreover, the dataset contains programs featuring both single and multiple faults. We utilized only valid test cases for the experiments, and the test suite included both passing and failing cases.

C. EVALUATION

We evaluated the effectiveness of the proposed method using two widely recognized metrics: *EXAM*score and *TOP-N* ranking. *EXAM*score quantifies the proportion of code that must be reviewed to locate a fault, with lower values indicating superior fault localization accuracy, as they suggest a reduction in the number of code sections requiring examination. The *TOP-N* ranking reflects the number of faulty code instances for which the fault localization technique successfully located the fault within the *TOP-N* positions [33]. In this analysis, the *TOP-N* ranking includes *Top-1*, *Top-3*, and *Top-5*, representing whether the fault was detected in the first, third, or fifth rank, respectively. In cases where multiple faults are present within a single program, only the first identified fault is considered for evaluation. The evaluation setup utilized a dataset comprising 6 programs from the *Code4Bench* benchmark, including 22 faulty versions and 65 distinct faults.

VI. RESULTS

In this section, we present an analysis of the results and address the research questions outlined in the study. Table 5 displays the correlation coefficient values for all features, including “Test Case Weight”, derived from logistic regression analysis to address the first research question. These results pertain to the experiment conducted using the proposed method with test case reduction, as specified in the first research question. For clarity, Table 5 provides the correlation coefficients for all sampling techniques employed in the study. These values highlight the relative importance of each feature in the analysis. The abbreviations used in Table 5 are defined as follows:

EN: EditedNearestNeighbours

RU: Random UnderSampler

RO: Random OverSampler

As shown in Table 5, the “Test Case Weight” emerges as the second most important feature (after “Change”) according to three sampling techniques, whereas it is ranked as the

TABLE 5. Logistic Regression Correlation Coefficients for the experiment with test case reduction.

	EN	RU	RO	SMOTE
Coverage	-0.002	0.002	0.005	-0.021
Data Dependency (<i>def</i>)	0.002	0.009	0.005	0.004
Data Dependency (<i>use</i>)	0.015	0.045	0.031	0.035
Control Dependency (<i>dom</i>)	-0.005	-0.024	-0.011	-0.015
Control Dependency (<i>domed</i>)	0.001	-0.024	-0.01	-0.008
Change	0.153	1.81	1.435	0.051
Test Case Weight	0.06	0.084	0.075	0.081

most important feature (with “Change” as the second) in one technique. In summary, among the five major features analyzed, “Test Case Weight” consistently demonstrates greater significance than program dependencies that are commonly utilized in fault localization. Furthermore, “Test Case Weight” is more informative than “Coverage” in fault localization, as it provides richer insights into the statements under examination.

Notably, “Change” consistently ranks as the most important feature by a significant margin. This can be attributed to the fact that any modification in the code has the potential to introduce faults. Despite this, “Change” is often overlooked in existing fault localization techniques. The correlation coefficient values further reveal that “Coverage” is not as strong a feature as traditionally assumed. In most sampling techniques, even “Data Dependency” holds greater value than “Coverage,” despite the latter being more widely used in practice. This discrepancy may explain the limitations observed in SBFL techniques. Regarding data dependency, “Data Dependency (*use*)” is distinctly more significant than “Data Dependency (*def*).” This is because, even if a value is incorrectly defined, the fault only becomes apparent when the value is utilized. Thus, “Data Dependency (*use*)” has a more substantial impact on fault localization.

The second research question is similar to the first, with the key difference being that test case selection was applied instead of test case reduction in the experiment. Consequently, Table 6 presents the correlation coefficient values for various sampling techniques derived from data collected

TABLE 6. Logistic Regression Correlation Coefficients for the experiment with test case selection.

	EN	RU	RO	SMOTE
Coverage	-0.011	-0.014	-0.013	-0.045
Data Dependency (<i>def</i>)	0.006	0.013	0.008	0.007
Data Dependency (<i>use</i>)	0.012	0.019	0.034	0.039
Control Dependency (<i>dom</i>)	-0.003	-0.011	-0.013	-0.018
Control Dependency (<i>domed</i>)	0.009	-0.019	-0.007	-0.002
Change	1.739	1.528	1.797	0.426
Test Case Weight	0.09	0.085	0.099	0.11

TABLE 7. Results for fault localization with test reduction.

	Top-1	Top-3	Top-5	Exam<5	Exam<10	Exam<15	Exam<20	Max exam	Min exam
Proposed method + EN	36.37%	63.64%	81.82%	36.37 %	50%	77.28%	81.82%	73.34	2.5
Proposed method + RU	36.37%	68.19%	81.82%	36.37 %	63.64%	77.28%	86.37%	34.48	2.5
Proposed method + RO	45.46%	68.19%	90.91%	45.46%	63.64%	81.82%	95.46%	24.14	2.5
Proposed method + SMOTE	22.73%	63.64%	81.82%	22.73%	50%	77.28%	81.82%	83.34	2.5
Tarantula	27.28%	40.91%	45.46%	27.28%	36.37%	40.91%	45.46%	68.96	2.5
Jaccard	27.28%	40.91%	54.55%	27.28%	36.37%	54.55%	54.55%	62.07	2.5
Ochiai	22.73%	36.37%	50%	22.73%	31.82%	50%	50%	71.43	2.5

when test case selection was implemented using the proposed method.

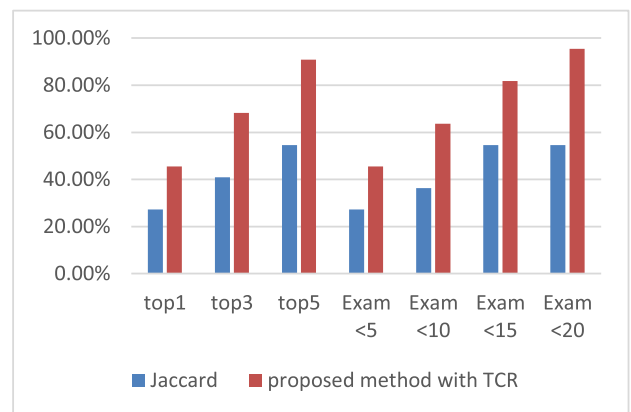
As evident from Table 6, the “Test Case Weight” consistently ranks as the second most important feature across all four sampling techniques. In this experiment, “Change” remains the most significant feature without exception. To summarize, “Change” is by far the most critical feature. At the same time, “Test Case Weight” predominantly occupies the second position in terms of importance, with a substantial margin in correlation coefficient values compared to “Coverage” and dependencies. Despite this, “Change” is rarely utilized in fault localization techniques, and “Test Case Weight” is often overlooked in favor of more conventional features such as “Coverage” and dependencies. Additionally, “Data Dependency (use)” provides more meaningful insights for fault localization than “Data Dependency (def).” Similarly, “Control Dependency (dom)” holds greater value than “Control Dependency (domed),” as the act of dominating is more influential than being dominated and can more effectively guide the fault localization process.

To address the third and fourth research questions, Table 7 presents the Top-1, Top-3, and Top-5 rankings, as well as the EXAM scores for 5, 10, 15, and 20, using each sampling technique. This table reflects the experiment results conducted with test case reduction applied. Additionally, it includes the results for SBFL for the same metrics, enabling a direct comparison between the proposed method and these established techniques. In this experiment, the proposed method uses a maximum of 80%, a minimum of 40%, and an average of 63% of the test suite to locate faults.

As demonstrated in Table 7, the proposed method significantly outperforms the SBFL formulae. When comparing the best sampling technique in the proposed method—*RandomOverSampler (RO)*—with the best-performing SBFL formula—*Jaccard*—the proposed method identifies 18.18% more faults at the *Top-1* ranking. It also detects 27.28% more faults at the *Top-3* ranking and 36.36% more at the *Top-5*. In terms of the *EXAM* score, the proposed method locates 45.46% of faults by examining a maximum of 5% of the code, 63.64% of faults by examining up to 10% of the code,

and 81.82% of faults by examining up to 15% of the code. Furthermore, it identifies 95.46% of faults (21 out of 22 faults) by analyzing a maximum of 20% of the code. According to Table 7, the highest *EXAM* score required to locate all faults is 24%, representing a significant improvement compared to *Jaccard*. Specifically, the proposed method outperforms *Jaccard* by identifying 18.18% more faults when examining 5% of the code, 27.27% more faults when examining 10% and 15%, and 40.91% more faults when examining 20% of the code.

The results suggest that the accuracy of fault localization can be enhanced by assigning greater prominence to features such as “Change” and “Test Case Weight.” Fig. 2 visually compares the proposed method with *Jaccard*, further illustrating these findings.

**FIGURE 2. Comparing the proposed method with TCR and Jaccard in accuracy and expense metrics.**

As shown in Fig. 2, the proposed method employing the TCR technique significantly outperforms Jaccard, the most effective of the three tested formulae, across all evaluation metrics, while utilizing a substantially smaller test suite.

To address the third research question, which involves test case selection, as well as the fifth research question, Table 8 provides the corresponding values for accuracy and computational expense, structured similarly to Table 7.

TABLE 8. Results for fault localization with test case selection.

	Top-1	Top-3	Top-5	Exam<5	Exam<10	Exam<15	Exam<20	Max exam	Min exam
Proposed method + EN	36.37%	81.82%	90.91%	40.91%	68.19%	90.91%	90.91%	44.83	2.5
Proposed method + RU	27.28%	59.1%	77.28%	27.28%	59.1%	72.73%	77.28%	72.41	3.22
Proposed method + RO	36.37%	72.73%	95.46%	40.91%	68.19%	81.82%	95.46%	27.59	2.5
Proposed method + SMOTE	36.37%	63.64%	81.82%	36.37%	50%	77.28%	81.82%	50	2.5
Tarantula	13.64%	27.28%	40.91%	13.64%	27.28%	27.28%	40.91%	73.08	2.5
Jaccard	18.19%	36.37%	50%	18.19%	31.82%	40.91%	50%	66.67	2.5
Ochiai	13.64%	27.28%	45.46%	13.64%	27.28%	36.37%	45.46%	71.43	2.5

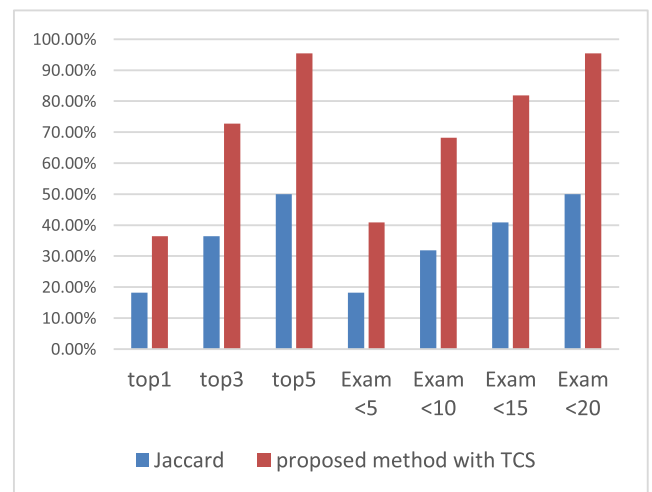
As evidenced in Table 8, the proposed method significantly outperforms all SBFL formulae across various sampling techniques by strategically selecting test cases that prioritize changes and faults. When comparing the best-performing sampling technique in the proposed method—*RandomOverSampler (RO)*—with the top SBFL technique—*Jaccard*—the proposed method identifies 18.18% more faults at the *Top-1* ranking, 36.36% more faults at the *Top-3* ranking, and 45.46% more faults at the *Top-5* ranking. Furthermore, the proposed method achieves complete fault localization by examining a maximum of 28% of the code, substantially improving over *Jaccard*. In terms of the *EXAM* score, the proposed method detects 22.72% more faults with an *EXAM* score of 5, 36.37% more faults with an *EXAM* score of 10, 40.91% more faults with an *EXAM* score of 15, and 45.46% more faults by analyzing only 20% of the code. These results underscore the critical importance of integrating “Change” and “Test Case Weight” with traditional features such as “Coverage” and Dependencies for effective fault localization. These findings were achieved under circumstances where the proposed method with test case selection utilizes a maximum of 68%, a minimum of 40%, and an average of 51% of the test suite for fault localization, demonstrating its efficiency.

A key strength of the proposed method lies in its ability to maintain high accuracy while significantly reducing the size of the test suite. In contrast, the performance of SBFL formulae deteriorates as the number of test cases is reduced, as evidenced by the comparison between test case selection and test case reduction results. Fig. 3 visually illustrates these findings in a chart format, clearly comparing the proposed method with *Jaccard*.

As illustrated in Fig. 3, the proposed method utilizing the TCS technique significantly exceeds *Jaccard* across all evaluation metrics.

VII. DISCUSSION

The proposed method employs five key features—“Coverage”, “Data Dependency (*def-use*)”, “Control

**FIGURE 3.** Comparing the proposed method with TCS and Jaccard in accuracy and expense metrics.

Dependency (*dom-domed*)”, “Change”, and “Test Case Weight”—to facilitate fault localization. Additionally, it integrates Test Case Reduction/Selection to optimize the test suite by either eliminating redundant test cases or selecting the most valuable ones. Findings suggest that “Control Dependency” contributes minimally to fault localization, whereas “Data Dependency” plays a more significant role, implying that a combination of “Data Dependency” and other features may be sufficient for effective fault localization.

Moreover, this study demonstrates that whereas SBFL utilizes essential fault localization data, it remains insufficient. “Test Case Weight”, derived from the same data but through a different approach, proves to be more influential than mere “Coverage”. Notably, this feature is entirely shaped by regression testing techniques, reinforcing the idea that regression testing could play a crucial role in fault localization for evolving software. However, unlike this study, the application of regression testing in fault localization from this perspective has received little attention. Additionally, program modifications emerge as strong indicators for identifying faults.

Thus, this research highlights the overlooked yet valuable information embedded within evolutionary software, emphasizing its significance in fault localization.

VIII. CONCLUSION

The increasing reliance on information technology has heightened the demand for software systems and their continuous updates. Certain portions of the code change with each update, which may introduce faults in the software. Regression testing techniques, combined with automated fault localization methods, have been developed to efficiently and swiftly detect and pinpoint these faults, enabling rapid repair of the software's new version. Despite introducing and exploring numerous fault localization techniques, significant challenges persist, including low accuracy and suboptimal performance in specific environments. Consequently, this study aims to address the accuracy limitations prevalent in many fault localization techniques, including SBFL, by integrating dynamic features such as code coverage, dynamic data and control dependencies, historical change data from the versions of the program, and test case weight derived from regression testing techniques. The proposed method was also evaluated with two regression testing strategies—test case reduction and test case selection—to assess its performance under each approach. The results indicate that the proposed method can reduce the test suite size by an average of 63% and 50%, respectively, while improving fault localization accuracy. Specifically, it achieves an 18%-45% increase in fault detection for Top-1, Top-3, and Top-5 rankings and a 22-45% improvement in fault detection when considering EXAM scores for 5% to 20% of the code.

For future research, the effectiveness of the features utilized in this study for fault localization could be explored in the context of regression testing techniques. Specifically, an optimal combination of features that are beneficial for both fault localization and regression testing could be identified, potentially enhancing the efficiency and accuracy of both processes. Moreover, test case reduction and selection techniques can be optimized to enhance the accuracy of fault localization.

IX. THREATS TO VALIDITY

Several factors may affect the validity of our findings. The validity may be influenced by the relatively small size of the benchmark used in this study, both in terms of source codes and the associated test suite. This limitation may lead to bias in the results, as smaller programs may not fully capture the complexity and diversity of real-world software systems. To address this, future work should evaluate the method on broader benchmark suites that vary in size, domain, and fault types. Moreover, a number of features are collected individually, which makes data collection and preprocessing both time- and resource-intensive. While we have taken steps to mitigate these threats, such as using consistent extraction procedures and automated scripts where possible, further

large-scale validation is necessary to strengthen confidence in the results.

REFERENCES

- [1] J. A. Jones and M. J. Harrold, "Empirical evaluation of the tarantula automatic fault-localization technique," in *Proc. 20th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Nov. 2005, pp. 273–282.
- [2] R. Abreu, P. Zoetewij, and A. J. C. Van Gemund, "An evaluation of similarity coefficients for software fault localization," in *Proc. 12th Pacific Rim Int. Symp. Dependable Comput. (PRDC)*, Dec. 2006, pp. 39–46. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/4041886>
- [3] W. E. Wong, V. Debroy, R. Gao, and Y. Li, "The DStar method for effective software fault localization," *IEEE Trans. Rel.*, vol. 63, no. 1, pp. 290–308, Mar. 2014, doi: [10.1109/TR.2013.2285319](https://doi.org/10.1109/TR.2013.2285319).
- [4] M. Wen, J. Chen, Y. Tian, R. Wu, D. Hao, S. Han, and S.-C. Cheung, "Historical spectrum based fault localization," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2348–2368, Nov. 2021, doi: [10.1109/TSE.2019.2948158](https://doi.org/10.1109/TSE.2019.2948158).
- [5] Z. Cui, M. Jia, X. Chen, L. Zheng, and X. Liu, "Improving software fault localization by combining spectrum and mutation," *IEEE Access*, vol. 8, pp. 172296–172307, 2020, doi: [10.1109/ACCESS.2020.3025460](https://doi.org/10.1109/ACCESS.2020.3025460).
- [6] Y. Li, S. Wang, and T. Nguyen, "Fault localization with code coverage representation learning," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, May 01, 2021, pp. 661–673. Accessed: Jan. 5, 2023. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9402079>
- [7] A. Dutta, S. S. Srivastava, S. Godbole, and D. P. Mohapatra, "Combi-FL: Neural network and SBFL based fault localization using mutation analysis," *J. Comput. Lang.*, vol. 66, Oct. 2021, Art. no. 101064, doi: [10.1016/j.cola.2021.101064](https://doi.org/10.1016/j.cola.2021.101064).
- [8] A. Maru, A. Dutta, K. V. Kumar, and D. P. Mohapatra, "Software fault localization using BP neural network based on function and branch coverage," *Evol. Intell.*, vol. 14, no. 1, pp. 87–104, Nov. 2019, doi: [10.1007/s12065-019-00318-2](https://doi.org/10.1007/s12065-019-00318-2).
- [9] A. A. Seyam, A. Hamdy, and M. S. Farhan, "Code complexity and version history for enhancing hybrid bug localization," *IEEE Access*, vol. 9, pp. 61101–61113, 2021, doi: [10.1109/ACCESS.2021.3074266](https://doi.org/10.1109/ACCESS.2021.3074266).
- [10] Y.-H. Li, J.-C. Chen, and Y.-Q. Zhao, "An effective fault localization approach based on weighted test cases," in *Proc. Comput. Sci. Artif. Intell.*, Aug. 2017, pp. 339–345, doi: [10.1142/9789813220294_0044](https://doi.org/10.1142/9789813220294_0044).
- [11] Y. Liu, J. Zhang, P. Nie, M. Gligoric, and O. Legunsen, "More precise regression test selection via reasoning about semantics-modifying changes," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 664–676, doi: [10.1145/3597926.3598086](https://doi.org/10.1145/3597926.3598086).
- [12] A. Dutta, "Enhancing fault localization by incorporating statement frequency and test case contribution," in *Proc. IEEE 23rd Int. Conf. Softw. Qual., Rel., Secur. (QRS)*, Oct. 2023, pp. 128–137.
- [13] D. Callaghan and B. Fischer, "Improving spectrum-based localization of multiple faults by iterative test suite reduction," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 1445–1457.
- [14] Y. Li, S. Wang, and T. N. Nguyen, "Fault localization to detect co-change fixing locations," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2022, pp. 659–671, doi: [10.1145/3540250.3549137](https://doi.org/10.1145/3540250.3549137).
- [15] F. Khorram, E. Bousse, A. Garmendia, J.-M. Mottu, G. Sunyé, and M. Wimmer, "From coverage computation to fault localization: A generic framework for domain-specific languages," in *Proc. 15th ACM SIGPLAN Int. Conf. Softw. Lang. Eng.*, Nov. 2022, pp. 235–248, doi: [10.1145/3567512.3567532](https://doi.org/10.1145/3567512.3567532).
- [16] M. Rezaalipour and C. A. Furia, "An empirical study of fault localization in Python programs," *Empirical Softw. Eng.*, vol. 29, no. 4, Jun. 2024, Art. no. 92, doi: [10.1007/s10664-024-10475-3](https://doi.org/10.1007/s10664-024-10475-3).
- [17] R. Widayarsi, G. A. A. Prana, S. A. Haryono, S. Wang, and D. Lo, "Real world projects, real faults: Evaluating spectrum based fault localization techniques on Python projects," *Empirical Softw. Eng.*, vol. 27, no. 6, Aug. 2022, Art. no. 147, doi: [10.1007/s10664-022-10189-4](https://doi.org/10.1007/s10664-022-10189-4).
- [18] J. Qian, X. Ju, and X. Chen, "GNet4FL: Effective fault localization via graph convolutional neural network," *Automated Softw. Eng.*, vol. 30, no. 2, Apr. 2023, Art. no. 16, doi: [10.1007/s10515-023-00383-z](https://doi.org/10.1007/s10515-023-00383-z).
- [19] Z. Li, E. Tang, X. Chen, L. Wang, and X. Li, "Graph neural network based two-phase fault localization approach," in *Proc. 13th Asia-Pacific Symp. Internetware*, Jun. 2022, pp. 85–95, doi: [10.1145/3545258.3545266](https://doi.org/10.1145/3545258.3545266).

- [20] R. Widyasari, G. A. A. Prana, S. A. Haryono, Y. Tian, H. N. Zachary, and D. Lo, "XA14FL," in *Proc. 30th IEEE/ACM Int. Conf. Program Comprehension*, May 2022, pp. 499–510, doi: [10.1145/3524610.3527902](https://doi.org/10.1145/3524610.3527902).
- [21] Y. Lei, H. Xie, T. Zhang, M. Yan, Z. Xu, and C. Sun, "Feature-FL: Feature-based fault localization," *IEEE Trans. Rel.*, vol. 71, no. 1, pp. 264–283, Mar. 2022, doi: [10.1109/TR.2022.3140453](https://doi.org/10.1109/TR.2022.3140453).
- [22] I. Ghani, W. M. N. Wan-Kadir, A. Firdaus Arbain, and N. Ibrahim, "Improved test case selection algorithm to reduce time in regression testing," *Comput., Mater. Continua*, vol. 72, no. 1, pp. 635–650, 2022, doi: [10.32604/cmc.2022.025027](https://doi.org/10.32604/cmc.2022.025027).
- [23] R. Ozkan, Z. Orman, and R. Samli, "A novel regression test selection method with graph-based genetic algorithm," *İleri Mühendislik Çalışmaları Ve Teknolojileri Dergisi*, vol. 4, no. 1, pp. 1–12, Aug. 2023. [Online]. Available: <https://dergipark.org.tr/tr/pub/imctd/issue/79680/1280717>
- [24] A. Mori, A. C. R. Paiva, and S. R. S. Souza, "An approach to regression testing selection based on code changes and smells," in *Proc. 8th Brazilian Symp. Syst. Automated Softw. Test.*, Brazil, Sep. 2023, pp. 25–34, doi: [10.1145/3624032.3624036](https://doi.org/10.1145/3624032.3624036).
- [25] S. Singhal, N. Jatana, A. F. Subahi, C. Gupta, O. I. Khalaf, and Y. Alotaibi, "Fault coverage-based test case prioritization and selection using African buffalo optimization," *Comput., Mater. Continua*, vol. 74, no. 3, pp. 6755–6774, 2023, doi: [10.32604/cmc.2023.032308](https://doi.org/10.32604/cmc.2023.032308).
- [26] S. K. Raju, S. Gopalan, S. K. Towfek, A. Sukumar, D. S. Khafaga, H. K. Alkahtani, and T. J. Alahmadi, "Test case selection through novel methodologies for software application developments," *Symmetry*, vol. 15, no. 10, p. 1959, Oct. 2023, doi: [10.3390/sym15101959](https://doi.org/10.3390/sym15101959).
- [27] S. Nagy, H. Amin, and N. Badr, "Enhanced regression testing execution process using test suite reduction techniques and parallel execution," *Int. J. Intell. Comput. Inf. Sci.*, vol. 23, no. 4, pp. 33–49, Dec. 2023, doi: [10.21608/ijicis.2023.243991.1303](https://doi.org/10.21608/ijicis.2023.243991.1303).
- [28] M. Singh, N. Chauhan, and R. Popli, "Test case reduction and SWOA optimization for distributed agile software development using regression testing," *Multimedia Tools Appl.*, vol. 84, no. 10, pp. 7065–7090, Apr. 2024, doi: [10.1007/s11042-024-19148-1](https://doi.org/10.1007/s11042-024-19148-1).
- [29] R. Pan, T. A. Ghaleb, and L. C. Briand, "LTM: Scalable and black-box similarity-based test suite minimization based on language models," *IEEE Trans. Softw. Eng.*, vol. 50, no. 11, pp. 3053–3070, Nov. 2024, doi: [10.1109/TSE.2024.3469582](https://doi.org/10.1109/TSE.2024.3469582).
- [30] H. He, J. Ren, G. Zhao, and H. He, "Enhancing spectrum-based fault localization using fault influence propagation," *IEEE Access*, vol. 8, pp. 18497–18513, 2020, doi: [10.1109/ACCESS.2020.2965139](https://doi.org/10.1109/ACCESS.2020.2965139).
- [31] J. Chu, T. Yu, J. Huffman Hayes, X. Han, and Y. Zhao, "Effective fault localization and context-aware debugging for concurrent programs," *Softw. Test., Verification Rel.*, vol. 32, no. 1, Oct. 2021, doi: [10.1002/stvr.1797](https://doi.org/10.1002/stvr.1797).
- [32] Y. Lou, Q. Zhu, J. Dong, X. Li, Z. Sun, D. Hao, L. Zhang, and L. Zhang, "Boosting coverage-based fault localization via graph-based representation learning," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2021, pp. 664–676, doi: [10.1145/3468264.3468580](https://doi.org/10.1145/3468264.3468580).
- [33] R. Widyasari, J. W. Ang, T. G. Nguyen, N. Sharma, and D. Lo, "Demystifying faulty code: Step-by-step reasoning for explainable fault localization," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, Mar. 2024, pp. 568–579, doi: [10.1109/SANER60148.2024.00064](https://doi.org/10.1109/SANER60148.2024.00064).



FAEZE AGHAZADE-PAR received the M.S. degree in information technology engineering from Shahid Beheshti University, Tehran, Iran, where she is currently pursuing the Ph.D. degree in software engineering. Her research interests include software debugging and gamification.



MOJTABA VAHIDI-ASL received the B.S. degree in computer engineering from the Amirkabir University of Technology and the M.S. and Ph.D. degrees in software engineering from Iran University of Science and Technology. He is currently an Assistant Professor with the Faculty of Computer Science and Engineering, Shahid Beheshti University, Tehran, Iran. His research interests include program analysis, software testing, and debugging.

• • •